

2.7 Trace data retrieval in Array, List and Queue

ایرے میں اگر انڈیکس معلوم ہو تو سیدھی چھلانگ لگتی ہے، جیسے $\text{Array}[2] = 20$ ۔

CS XII

Chapter 2: Computational Thinking & Algorithms

Lecture 23 of 24

2.7 Trace data retrieval in Array, List and Queue

LEARN IT

Tracing data retrieval means following the step-by-step path of how an algorithm finds or accesses data. Array, linked list and queue are all linear, but the **effort** to find an item is very different because of how they are organised inside.

2.7.1 Retrieval in an array

Every element has a fixed index from 0 to $n-1$ pointing at a spot in memory. If the algorithm already knows the index, it does not search: it **jumps** (direct access, $O(1)$).

Array = [10, 50, 20, 70, 90, 30]. Direct access: $\text{Array}[2] = 20$.

Loop iteration	Index	Array access	Value
1	0	Array[0]	10
2	1	Array[1]	50
3	2	Array[2]	20

```
Array = [10, 50, 20, 70, 90, 30]
print("Direct Array[2] =", Array[2])
for i in range(3):
    print(i + 1, i, f"Array[{i}]", Array[i])
```

OUTPUT

```
Direct Array[2] = 20
1 0 Array[0] 10
2 1 Array[1] 50
3 2 Array[2] 20
```

2.7.2 Retrieval in a linked list

Linked lists have **no index**. The only way is to start at HEAD and follow links one node at a time (sequential access, $O(n)$).

List: [12] → [98] → [23] → [87] → [56] → NULL. Search key: 87.

Step	Node data	Compare with 87	Action
1	12	$12 \neq 87$	Move next
2	98	$98 \neq 87$	Move next
3	23	$23 \neq 87$	Move next
4	87	$87 == 87$	FOUND — output True

```

nodes = [12, 98, 23, 87, 56]
key = 87
found = False
for i, value in enumerate(nodes, start=1):
    print(f"Step {i}: Check Node {i} ({value}).")
    if value == key:
        found = True
        print("FOUND! Output: True")
        break
    print("Not equal. Move Next.")
print("Result:", found)

```

OUTPUT

```

Step 1: Check Node 1 (12).
Not equal. Move Next.
Step 2: Check Node 2 (98).
Not equal. Move Next.
Step 3: Check Node 3 (23).
Not equal. Move Next.
Step 4: Check Node 4 (87).
FOUND! Output: True
Result: True

```

Key 87 sits at the fourth node, so you paid four checks. An array that already knew index 3 would have paid one jump.

2.7.3 Retrieval in a queue

A queue is FIFO. You **cannot** retrieve from the middle or the rear. The oldest item at the Front is always DEQUEUEED first.



You cannot pull B until A has left.

Queue A, B, C. ENQUEUE D joins the rear. DEQUEUE must remove A, not C or B.

```

from collections import deque
q = deque(["A", "B", "C"])
q.append("D")
print("after ENQUEUE D:", list(q))
print("DEQUEUE:", q.popleft())
print("now:", list(q))

```

OUTPUT

```

after ENQUEUE D: ['A', 'B', 'C', 'D']
DEQUEUE: A
now: ['B', 'C', 'D']

```

WORKED EXAMPLE

Worked example: three retrievals compared for "get 20 / get 87 / get A".

Structure	Target	Path	Can you jump?
Array [10, 50, 20, ...]	20 at index 2	Array[2] in one step, or scan 10, 50, 20	Yes, if the index is known
Linked list 12→98→23→87→...	87	Must visit 12, 98, 23, then 87	No
Queue A-B-C	A (the only legal retrieval)	DEQUEUE Front	No — never the middle

REMEMBER IT

Array jumps, list walks, queue waits. If the exam asks "why is array retrieval different?", write **direct access by index** versus **sequential links** versus **Front only**.

COMMON MISTAKE

Mistake: DEQUEUEing C because "C is the interesting letter", or reading linked-list node 4 as `list[3]` without walking.

Fix: show every hop in the table. Marks are for the path, not only for FOUND.

اردو میں سمجھیں

ایرے میں اگر انڈیکس معلوم ہو تو سیدھی چھلانگ لگتی ہے، جیسے $\text{Array}[2] = 20$ ۔ لنکڈ لسٹ میں HEAD سے ایک ایک نوڈ دیکھنا پڑتا ہے؛ 87 چوتھے نوڈ پر ملا۔ کیوں صرف Front نکلتا ہے، درمیان سے نہیں؛ A, B, C میں DEQUEUE ہمیشہ A ہے۔

CHECK YOURSELF

1. What is $\text{Array}[2]$ in $[10, 50, 20, 70, 90, 30]$?
2. How many nodes are checked before 87 is found in $12 \rightarrow 98 \rightarrow 23 \rightarrow 87 \rightarrow 56$?
3. In queue A, B, C, can you retrieve B without removing A?

Answers: 1. 20. 2. Four (12, 98, 23, 87). 3. No — a queue only DEQUEUEs the Front, so A must leave first.

Next lecture: Career Opportunities in Algorithms